# نامه درخواست استقرار Kubernetes — پروژه SMART / VOC Panel

**مخاطب:** تیم DevOps / زیر زیر **فرستنده:** تیم توسعه (Backend + Frontend) **موضوع:** استقرار دو سرویس Frontend و Backend پروژه داشبورد SMART روی Kubernetes **تاریخ:** ۱۸/۰۸/۱۴۰۴

با سلام و احترام،

لطفاً دو اپلیکیشن زیر را مطابق الگوی استقرار فعلی شرکت (مشابه Helm Chart سوپرست و استقرار سرویس‌هایی مثل Kernoon با RabbitMQ / Redis / Postgres) روی کلاستر Kubernetes آماده‌سازی و Deploy کنید.

## ۱) معرفی سرویس‌ها

| سرویس | ریپازیتوری | تکنولوژی | پورت داخل کانتینر | توضیح |
|---|---|---|---|---|
| **Backend API** | `voc_panel` | NestJS + Fastify + TypeORM | `5001` | API احراز هویت، تیکت، همگام‌سازی CRM |
| **Frontend** | `smart-dashboard` | React (Vite) + Nginx | `80` | داشبورد SMART؛ در بیلد به API وصل می‌شود |

### وابستگی‌های Backend (مشابه اکوسیستم فعلی)

| وابستگی | نسخه پیشنهادی | پورت داخلی | توضیح |
|---|---|---|---|
| PostgreSQL | `17-alpine` | `5432` | دیتابیس اصلی تیکت/کاربر |
| Redis | `8.10.2` (یا `8-alpine`) | `6379` | کش / session کمکی |
| RabbitMQ | `4-management-alpine` | `5672` (+ `15672` management اختیاری) | صف پیام |

> در محیط لوکال همین وابستگی‌ها با `docker-compose.yml` داخل `voc_panel` بالا می‌آیند. در K8s ترجیح ما استفاده از سرویس‌های managed/موجود کلاستر است؛ در غیر این صورت StatefulSet / Helm chart مشابه Postgres و Redis سوپرسِت.

---

## ۲) وضعیت فعلی آماده‌سازی توسط تیم توسعه

### Frontend (`smart-dashboard`)

- Dockerfile آماده است (multi-stage: build `node:22-alpine` → `nginx:1.27-alpine`).
- Build arg مهم: `VITE_API_BASE_URL` باید آدرس عمومی/داخلی Backend در کلاستر باشد مثال: `https://voc-api.<domain>` یا `http://voc-panel-api.<namespace>.svc.cluster.local:5001`
- Nginx برای SPA با `try_files` پیکربندی شده است.

### Backend (`voc_panel`)

- کد code-first است؛ با `TYPEORM_SYNCHRONIZE=true` در غیرپروداکشن جداول ساخته می‌شود.
- **هنوز Dockerfile اختصاصی در ریپو ندارد** — لطفاً یا شما بسازید، یا به ما بگویید طبق استاندارد ایمیج‌های Node شرکت (مشابه `kernoon-back`) بسازیم.
- فایل env نمونه: `.sample.env` / `.prod.env`

### نیاز شبکه‌ای خاص (خیلی مهم)

Backend برای Job همگام‌سازی تیکت باید به CRM داخلی شرکت دسترسی داشته باشد:

- `https://crm.dayins.com/panel/rest/voc_fetch_all/`
- `https://crm.dayins.com/panel/rest/voc_fetch_all_calls/`

این API فقط از شبکه شرکت / مسیر مجاز egress در دسترس است. لطفاً NetworkPolicy / NAT / Proxy لازم را طوری تنظیم کنید که Podهای Backend به `crm.dayins.com` برسند.

---

## ۳) درخواست مشخص از تیم DevOps

لطفاً موارد زیر را انجام دهید / نهایی کنید:

1. **Namespace** مناسب (پیشنهاد: `smart` یا `voc-panel`).
2. **Container Registry** و مسیر ایمیج‌ها (مثلاً `registry.../smart/voc-panel-api` و `registry.../smart/smart-dashboard`).
3. ساخت **Dockerfile Backend** (اگر در استاندارد شماست) + CI برای build/push.
4. ایجاد منابع K8s (مطابق الگوی Helm/سوپرست شرکت):

- `Deployment` برای API و Frontend
- `Service` (ClusterIP)
- `Ingress` + TLS
- `Secret` برای رمزها و JWT
- `ConfigMap` برای تنظیمات غیرحساس
- `HorizontalPodAutoscaler` در صورت نیاز
- `readinessProbe` / `livenessProbe`

1. تهیه Postgres / Redis / RabbitMQ در همان namespace یا اتصال به سرویس‌های اشتراکی موجود.
2. تنظیم DNS و CORS:

- Frontend origin باید در CORS بک‌اند مجاز باشد.
- `VITE_API_BASE_URL` در بیلد فرانت برابر URL عمومی API شود.

1. دسترسی egress به `crm.dayins.com` برای Job همگام‌سازی CRM.
2. اعلام آدرس نهایی Staging و Production به تیم توسعه برای تست.

---

## ۴) پیشنهاد نام‌گذاری منابع (قابل تغییر توسط شما)

```
Namespace:          smart

Backend Deployment: voc-panel-api

Backend Service:    voc-panel-api:5001

Frontend Deployment:smart-dashboard

Frontend Service:   smart-dashboard:80
```

```
Ingress hosts (نمونه):
  - smart.<domain>          → frontend
  - smart-api.<domain>      → backend
```

Service discovery داخل کلاستر (مشابه الگوی `.svc.cluster.local` در values سوپرست):

```
POSTGRES_HOST=voc-panel-postgres   (یا سرویس managed)
REDIS_HOST=voc-panel-redis
RABBITMQ_HOST=voc-panel-rabbitmq
```

پورت‌های داخلی سرویس‌ها باید پورت استاندارد کانتینر باشد (`5432` / `6379` / `5672`)، نه پورت‌های remap لوکال.

---

## ۵) متغیرهای محیطی مورد نیاز Backend

لطفاً این‌ها را در `ConfigMap` / `Secret` قرار دهید (مقادیر پروداکشن را خودتان تولید کنید):

```
NODE_ENV=prod
PORT=5001

POSTGRES_HOST=<k8s-service-name>
POSTGRES_PORT=5432
POSTGRES_DB=voc_panel
POSTGRES_USER=<secret>
POSTGRES_PASSWORD=<secret>
TYPEORM_SYNCHRONIZE=false

REDIS_HOST=<k8s-service-name>
```

```
REDIS_PORT=6379

REDIS_PASSWORD=<secret>



RABBITMQ_HOST=<k8s-service-name>

RABBITMQ_PORT=5672

RABBITMQ_USER=<secret>

RABBITMQ_PASSWORD=<secret>

RABBITMQ_VHOST=/



JWT_SECRET=<long-random>

JWT_REFRESH_SECRET=<long-random-different>



SUPERSET_URL=<url-superset-existing>

SUPERSET_USERNAME=<secret>

SUPERSET_PASSWORD=<secret>

SUPERSET_PROVIDER=db



CRM_FETCH_URL=https://crm.dayins.com/panel/rest/voc_fetch_all/

CRM_CALLS_FETCH_URL=https://crm.dayins.com/panel/rest/voc_fetch_all_calls/

CRM_SOURCE=smart

CRM_FETCH_LIMIT=1000

CRM_SYNC_ENABLED=true

CRM_INITIAL_LAST_TICKETID=0
```

### Frontend build-arg

```
VITE_API_BASE_URL=https://smart-api.<domain>
```

---

## ۶) Healthcheck پیشنهادی

| سرویس | Path / فرمان | توضیح |
|---|---|---|
| Backend | `GET /` روی پورت `5001` | readiness/liveness ساده |
| Frontend | `GET /` روی پورت `80` | Nginx |
| Postgres | `pg_isready` | مثل compose فعلی |
| Redis | `redis-cli ping` | با پسورد |
| RabbitMQ | `rabbitmq-diagnostics -q ping` | |

---

## ۷) نکات امنیتی و عملیاتی

1. در Production مقدار `TYPEORM_SYNCHRONIZE=false` باشد؛ schema با migration یا فرآیند مصوب شما اعمال شود.
2. Secretها را در Git نگذارید؛ از Sealed Secrets / External Secrets (همان الگوی `secretEnv` سوپرسِت) استفاده کنید.
3. Job همگام‌سازی CRM ممکن است طولانی باشد (chunkهای ۵۰۰تایی تا `has_more=false`)؛ timeout Ingress/Proxy برای مسیرهای `/crm/*` را مناسب تنظیم کنید (مثلاً چند دقیقه).
4. فقط نقش `admin` به `POST /crm/sync` دسترسی دارد.
5. CORS بک‌اند باید Origin فرانت نهایی را شامل شود.

---

## ۸) چک‌لیست تحویل از طرف DevOps به توسعه

پس از استقرار Staging لطفاً این‌ها را بفرستید:

- ☐ URL فرانت
- ☐ URL API + Swagger (`/docs`)
- ☐ وضعیت دسترسی Pod به `crm.dayins.com`
- ☐ نام namespace و ایمیج‌های نهایی
- ☐ نحوه مشاهده لاگ‌ها (`kubectl logs ...`)
- ☐ یوزر/پسورد یا روش دسترسی به Postgres استیجینگ برای smoke test

ما از سمت توسعه این‌ها را تست می‌کنیم: Login با موبایل، `GET /auth/me`، لیست تیکت، و در صورت VPN/`crm.dayins.com` دکمه «همگام‌سازی CRM» در تنظیمات ادمین.

---

## ۹) کارهایی که تیم توسعه انجام می‌دهد / آماده است

- کد Backend و Frontend آماده و روی Git است.
- Frontend Dockerfile + nginx config آماده است.
- Compose لوکال برای Postgres/Redis/RabbitMQ آماده است.
- در صورت نیاز، Dockerfile استاندارد Backend را طبق الگوی `kernoon-back` می‌سازیم و PR می‌زنیم.
- همکاری برای تنظیم env، CORS و smoke test بعد از Deploy.

---

پیشاپیش از همکاری شما سپاسگزاریم. در صورت نیاز به جلسه کوتاه برای هم‌راستا کردن namespace / Ingress / Registry در خدمتم.

با احترام، تیم توسعه SMART / VOC Panel

---

## پیوست فنی سریع برای DevOps

### معماری منطقی

```
[Browser]
   |
   ▼
[Ingress] ──► smart-dashboard (Nginx:80)
   |
   └──► voc-panel-api (Nest:5001)
              |
              ├── PostgreSQL
              ├── Redis
              ├── RabbitMQ
              └── egress ► crm.dayins.com  (VPN/شرکتی)
```

### منابع مرجع الگوی شرکت در همین ماشین توسعه

- Helm سوپرست: `Documents/superset/helm/superset/` (`deployment.yaml`, `ingress.yaml`, `values.yaml` با Postgres/Redis و `envFromSecret`)
- الگوی وابستگی‌ها: `Documents/kernoon-back/deploy/swarm/stack.yml` (Postgres 17 + RabbitMQ 4 + Redis 8)
- Frontend Docker: `Documents/smart-dashboard/Dockerfile`
- Backend compose لوکال: `Documents/voc_panel/docker-compose.yml`